

재입원 위험도(고위험, 중위험, 저위험) 예측

드레안 의료/헬스케어
 목 표: 재입원 위험도(고위험, 중위험, 저위험) 문제

서울에 위치한 D병원은 병원 환자들의 30일 이내 재입원율이 증가하는 문제에 직면해 있습니다.

재입원은 환자의 건강상태 악화를 의미할 뿐만 아니라, 병원의 의료 자원 운영에도 부담이 되고 있습니다.

이에 D병원은 환자의 재입원 위험도를 사전에 예측하여 고위험군 환자들에게 맞춤형 관리 프로그램을 제공하고자 합니다.

병원 데이터 등을 AI 기술로 분석하여, 퇴원 시점에서 30일 이내 재입원 가능성을 예측하고자 합니다.

본 인공지능은 병원 데이터를 분석하고 AI 모델을 통해, 퇴원 후 30일 이내 재입원 위험도(고위험, 중위험, 저위험)를 예측 분석하는 문제로 구성되어 있습니다.

[유의사항]

- 가 문항의 답안코드는 반드시 **부** 여기에 답안코드를 작성하세요'로 표시된 셀에 작성해야 합니다.
- 제출된 시험결과 셀을 삭제하거나 답안 위치가 아닌 다른 셀에 답안코드를 작성 시 해당되지 않습니다.
- 시험 중에는 상단의 '임시저장' 버튼을 클릭하여 저장을 해주시고, 답안 제출시에는 '최종제출' 버튼을 클릭해주시기 바랍니다.
- 반드시 문제에 제시된 가이드를 읽고 답안 작성하세요.
- 문제에 변수명이 제시된 경우 반드시 해당 변수명을 사용하세요.
- 자격인증 문제에 제공된 데이터는 제 3자에게 공유하거나 개인적인 용도로 사용하는 등 외부로 유출할 수 없으며 유출로 인한 책임은 응시자 본인에게 있습니다.
- 오른쪽 하단사이트바에만 답고 가능하며 그외 사이트 접근시 부정행위로 간주될수 있습니다.**

[데이터 실험 설명 (데이터 파일명: hospital_data_v2.csv)]

- P_ID : 환자ID
- PAT_TYPE : 재입원 위험도(고위험, 중위험, 저위험)
- DIAG_CAT : 진단 범주
- SEVERITY_LEVEL : 심각 수준
 - HOSP_ADDR_NO : 병원 호프명번호
- WARD_NO : 병동 번호
- ROOM_NO : 병실 번호
- age : 나이

다음 문항을 풀기 전에 아래 코드를 실행해주세요.

```

In [1]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

import matplotlib.font_manager as fm
plt.rc('font', family='NamGothicCoding')
```

1. Numpy는 데이터 분석을 위해 가장 많이 사용하는 파이썬 라이브러리입니다.

아래 가이드에 따라 올바른 선택지를 고르고, 선택한 코드를 셀에 작성하고 실행시키세요.

- Numpy를 np라는 별칭(alias)으로 불러오는 코드로 맞는 선택지를 골라 '답안01' 변수에 저장하세요.(예, 답안01 = 5)
 - import numpy as np
 - import numpy alias np
 - import np as numpy
 - import np alias numpy
- 선택한 선택지를 셀에 작성하고 실행시키세요.

여기에 선택한 코드를 작성하고 실행하세요. 그리고 답안01 변수의 답안코드를 작성하세요.

2. 모델링을 위해 분석 및 처리할 데이터 파일을 읽어오려고 합니다.

아래 가이드에 따라 올바른 선택지를 고르고, 선택한 코드를 셀에 작성하고 실행시키세요.

- pandas를 활용하여 'hospital_data_v2.csv' 데이터 파일을 읽고 df 데이터프레임 변수명에 저장하는 코드로 맞는 선택지를 고르세요.(예, 답안02 = 5)

입을 데이터 파일명 'hospital_data_v2.csv' csv 파일은 본 문제/답안지의 동일한 경로에 있습니다)

저장할 데이터프레임 변수명 : df

 - df = pd.read_json('hospital_data_v2.csv')
 - df = pd.read_csv('hospital_data_v2.csv')
 - df = pd.read_html('hospital_data_v2.csv')
 - df = pd.read_sql('hospital_data_v2.csv')
- 선택한 선택지를 셀에 작성하고 실행시키세요.

여기에 선택한 코드를 작성하고 실행하세요. 그리고 답안02 변수의 답안코드를 작성하세요.

3. 읽어들이는 데이터에 대한 기술적 통계를 파악하고자 합니다.

아래 가이드에 따라 실행하고 결과에 대해서 올바른 선택지 고르세요.

- 대상 데이터프레임 : df
- describe 함수를 실행하여 기술적 통계를 확인합니다.
- describe 실행 결과로 출처 않은 선택지를 골라 '답안03' 변수에 저장하세요.(예, 답안03 = 5)
 - 'AGE' 컬럼의 최소값은 30이다.
 - 'AGE' 컬럼의 최대값은 150이다.
 - 'AGE' 컬럼의 1분위값은 27이다.
 - 'WARD_NO' 컬럼의 3분위값은 174이다.

여기에 코드를 작성하고 실행하세요. 그리고 답안03 변수의 답안코드를 작성하세요.

4. 재입원 위험도에 대한 분포를 파악하고자 합니다.

Seaborn을 활용하여, '재입원 위험도'의 분포를 확인 하는 countplot을 그리세요.

- 대상 데이터프레임 : df
- 재입원 위험도 : PAT_TYPE
- Seaborn의 countplot 함수 활용하여 'PAT_TYPE' 컬럼에 대한 분포를 그리세요.
- 안내된 내용 외 별도의 파라미터를 입력하지 마세요.

여기에 코드를 작성하고 실행하세요.

5. AI모델링 결과의 왜곡을 초래하는 결측치에 대해서 확인하려고 합니다.

아래 가이드에 따라 올바른 선택지를 고르세요.

- 대상 데이터프레임 : df
- 아래 컬트 그림을 보시면 df 데이터프레임의 컬럼별 결측치의 합계를 보여주는 코드와 결과가 보입니다.
- 코드 결과를 보면 컬럼별로 결측치가 0으로 나오고 있습니다.
- 빈칸에 들어갈 알맞은 코드가 있는 선택지를 골라 '답안05' 변수에 저장하세요.(예, 답안05 = 5)
 - mulina()
 - nullna()
 - isnull()
 - isnull

```

df.[ 빈칸 ].sum()

P_ID      0
PAT_TYPE  0
DIAG_CAT  0
SEVERITY_LEVEL  0
HOSP_ADDR_NO  0
WARD_NO   0
ROOM_NO   0
AGE       0
dtype: int64
```

여기에 답안05 변수의 답안코드를 작성하세요.

6. 범주형 데이터를 수치형 데이터로 변환해주는 데이터 전처리 방법 중 하나로 label encoder를 사용합니다.

아래 가이드에 따라 범주형 데이터를 수치형 데이터로 변환하도록 빈칸에 알맞은 코딩을 작성하고 실행하세요.

- 가이드에 따라 라벨 인코딩 수행합니다
 - 대상 데이터프레임 : df
 - 라벨 인코딩 전처리 대상 컬럼: 'PAT_TYPE' (재입원 위험도)
 - LabelEncoding을 활용하여 범주형 데이터의 데이터를 수치형 데이터로 변환합니다.
- 아래 주어진 라벨 인코딩 코드의 빈칸을 채우고 실행하세요
 - 그리고, 빈칸에 들어갈 알맞은 코드를 '답안06' 변수에 저장하세요.(예, 답안06 = '알맞은코드 입력하세요')

여기에 주어진 코드의 빈칸을 채우고 실행하세요. 그리고 답안06 변수의 답안코드를 작성하세요.

```

df_LE = df.copy()
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df_LE['PAT_TYPE'] = le. < 빈칸 > (df_LE['PAT_TYPE'])
print(le.classes_)
```

다음 문항을 풀기 전에 아래 코드를 실행해주세요.

```

In [ ]: df.info()
```

7. 원-핫 인코딩(One-hot encoding)은 범주형 변수를 1과 0의 이진형 벡터로 변환하기 위하여 사용하는 방법입니다.

아래 가이드에 따라 수행하고 출처 않은 선택지를 고르세요.

- 가이드에 따라 원-핫 인코딩 수행합니다.
 - 대상 데이터프레임 : df_LE
 - 원-핫 인코딩 대상 컬럼
 - 'DIAG_CAT': 진단 범주
 - 'SEVERITY_LEVEL': 심각 수준
 - 원-핫 인코딩 함수를 Pandas의 get_dummies 사용합니다.
 - 원-핫 인코딩의 drop_first=True 옵션을 사용하여 기본값 1개를 삭제합니다.
 - 원-핫 인코딩의 dtype=int 옵션을 사용합니다.
- 아래 주어진 원-핫 인코딩 코드를 수행하세요.
 - 해당 전처리가 반영된 df_LE를 데이터프레임 변수 df_preset에 저장합니다.
- 아래 주어진 원-핫 인코딩 결과가 이전의 원-핫 출력과 원-핫 인코딩 수행 후의 info 출력을 비교합니다.
- 원-핫 인코딩 수행에 대해서 출처 않은 선택지를 골라 '답안07' 변수에 저장하세요.(예, 답안07 = 5)
 - 1개의 'DIAG_CAT' (진단 범주) 컬럼은 원-핫인코딩을 통해서 7개의 컬럼으로 변형되었다.
 - 1개의 'SEVERITY_LEVEL' (심각 수준) 컬럼은 원-핫인코딩을 통해서 5개의 컬럼으로 변형되었다.
 - 원-핫 인코딩의 drop_first=True 옵션이 주어지지 않았다면 'DIAG_CAT' (진단 범주) 컬럼은 원-핫인코딩을 통해서 6개의 컬럼으로 변형되었을 것이다.
- dtype=int 옵션을 사용하여 원-핫인코딩 결과가 True나 False가 아니라 1과 0으로 보기에 한다.

다음 문항을 풀기 전에 아래 코드를 실행해주세요.

```

In [ ]: df_preset = pd.get_dummies(data=df_LE, columns= ['DIAG_CAT', 'SEVERITY_LEVEL' ], drop_first=True, dtype=int)
df_preset.info()
```

여기에 답안07 변수의 답안코드를 작성하세요.

다음 문항을 풀기 전에 아래 코드를 실행해주세요.

```

In [ ]: from sklearn.model_selection import train_test_split

df_del = df_preset.drop(['P_ID', 'HOSP_ADDR_NO'], axis=1)

X = df_del.drop('PAT_TYPE', axis=1)
y = df_del['PAT_TYER']

X_train, X_valid, y_train, y_valid = train_test_split(X, y, test_size=0.3, stratify=y, random_state=42)
X_train.shape, X_valid.shape, y_train.shape, y_valid.shape
```

8. 랜덤포레스트(RandomForest)는 분류, 회귀 분석 등에 사용되는 앙상블 학습 방법입니다.

아래 가이드에 따라 작성한 랜덤포레스트 모델의 코드의 빈칸을 작성하고 실행하세요.

- 가이드에 따라 랜덤포레스트(RandomForest) 모델의 코드를 작성하였습니다.
 - 결정트리 의 개수(n_estimators)는 50로 설정합니다.
 - 최대 Feature 개수(max_features)는 10로 설정합니다.
 - 결과의 최대 깊이(max_depth)는 13로 설정합니다.
 - 리프노드가 되기 위해 필요한 최소한의 샘플 데이터수(min_samples_leaf)는 2로 설정합니다.
 - random_state는 30로 설정합니다.
 - 랜덤포레스트 모델을 rf 변수에 저장합니다.
 - 학습시 훈련데이터 셋을 활용해 모델을 학습합니다.
 - 학습된 모델로 검증데이터에 대해 예측을 수행합니다.
 - 학습된 모델로 검증데이터 셋에 대해 예측을 수행하고 y_pred에 저장합니다
- 아래 작성된 랜덤포레스트(RandomForest) 모델의 주어진 코드의 빈칸을 채우시고 실행하세요.
 - 그리고, 빈칸에 들어갈 알맞은 코드를 '답안08' 변수에 저장하세요.(예, 답안08 = '알맞은코드 입력하세요')

여기에 주어진 코드의 빈칸을 채우고 실행하세요. 그리고 답안08 변수의 답안코드를 작성하세요.

```

from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.metrics import accuracy_score

rf = RandomForestClassifier(n_estimators=50, max_features=10, max_depth=13, min_samples_leaf=2, random_state=30)
rf.fit(X_train, y_train)
y_pred = rf. < 빈칸 > (X_valid)
```

9. 위 랜덤포레스트 모델의 성능을 평가하려고 합니다.

아래 가이드에 따라 작성한 랜덤포레스트 모델의 성능을 확인하는 코드의 빈칸을 작성하고 실행하세요.

그리고, 아래 주어진 선택지 중에서 출처 않은 선택지를 고르세요.

- 앞에서 만든 랜덤포레스트 모델의 성능 확인하기 위해 가이드에 따라 코드를 작성하였습니다.
 - 이전 단계에서 만든 데이터에 대해 예측(y_pred)한 y_pred 값을 출력합니다.
 - 결측 데이터의 정답과 예측값을 가지고 Confusion matrix를 구하고 heatmap 그래프로 시각화합니다.
 - Sokli learn의 classification report 가 능을 사용하여 콜러스널 precision, recall, f1-score를 출력합니다.
- 아래 작성된 랜덤포레스트(RandomForest) 모델의 성능 확인 코드의 빈칸을 채우시고 실행하세요.
 - 그리고, 빈칸에 들어갈 알맞은 코드를 '답안09_1' 변수에 저장하세요.(예, 답안09_1 = '알맞은코드 입력하세요')
- 주어진 선택지를 읽고 출처 않은 선택지 골라 '답안09_2' 변수에 저장하세요.(예, 답안09_2 = 5)
 1. 고위험(고위험)의 정답도율은 98% 이다.
 2. 저위험(저위험)의 정답도율은 98% 이다.
 3. 중위험(중위험)의 F1-점수는 100% 이다.
 4. 정확도는 98% 이다.

여기에 주어진 코드의 빈칸을 채우고 실행하세요. 그리고 답안09_1, 답안09_2 변수의 답안코드를 작성하세요.

```

from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix

print(classification_report(y_valid, y_pred))
mtx = confusion_matrix(y_valid, y_pred)
sns. < 빈칸 > (mtx, annot=True, fmt = 'd')
```

10. 앞에서 학습한 머신러닝 모델으로 시뮬레이션 수행 및 변수중요도를 확인하고자 합니다.

아래 가이드에 따라 답안을 작성하세요.

- 앞에서 만든 랜덤포레스트 모델로 시뮬레이션 수행 및 변수중요도를 확인하기 위해 가이드에 따라 코드를 작성하였습니다.
 - 학습한 랜덤포레스트 모델 r을 활용하여 시뮬레이션 데이터(sim_data)를 예측(predict)을 수행합니다.
- 가이드에 따라 답안코드를 작성하세요.
 - 시뮬레이션 데이터(sim_data)를 예측값을 '답안10' 변수에 저장하세요.(예, 답안10 = 0 혹은 1 혹은 2)
- 랜덤포레스트 모델 r의 변수중요도 예측에서 '답안10' 변수의 변수중요도의 이름을 '답안10_1' 변수에 저장하세요.(예, 답안10_1 = '알맞은이름 입력하세요')

시뮬레이션 데이터와 변수중요도를 사용하기 위해 아래 코드를 실행하세요.

```

In [ ]: test_data = [ [244, 0, 17, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1] ]
print(rf.predict(test_data))
pd.Series(rf.feature_importances_, index=X.columns).sort_values().plot(kind='bar')
```

여기에 답안10 변수의 답안코드를 작성하세요.

1년부터 10년까지 모든 문제를 풀었습니다. 수고하셨습니다.

